

Comparative Analysis of L1 and L2 Regularization

Feature Selection in Housing Price Prediction

Project Report

Muhammad Anwar Raza (MSDS25002) - Team Lead

Fahad Khalid (MSDS25025)

Noman Ali (MSDS25054)

Course: CS587 - Statistical and Mathematical Tools for Data Science

Information Technology University (ITU), Pakistan

December 5, 2025

Abstract

This project compares L1 (Lasso) and L2 (Ridge) regularization techniques for housing price prediction using the Ames Housing dataset (2,930 observations, 82 features). We investigate how different penalty functions affect feature selection and predictive performance. Our results show that Lasso successfully eliminates 53.7% of features while maintaining test $R^2 = 0.8674$, comparable to Ridge's $R^2 = 0.8780$ using all features. The optimal regularization parameters are $\lambda_{Ridge} = 494.171$ and $\lambda_{Lasso} = 0.003$. This demonstrates that L1 regularization provides automatic feature selection with minimal performance loss, creating simpler, more interpretable models suitable for real-world deployment.

Keywords: L1 Regularization, L2 Regularization, Lasso, Ridge, Feature Selection, Housing Price Prediction

Contents

1	Introduction	4
1.1	Problem Statement	4
1.2	Objectives	4
1.3	Dataset	4
2	Theoretical Background	4
2.1	Linear Regression	4
2.2	Ridge Regression (L2)	5
2.3	Lasso Regression (L1)	5
2.4	Why L1 Induces Sparsity	5
2.5	Bayesian Interpretation	5
3	Methodology	5
3.1	Data Preprocessing	5
3.1.1	Missing Values	5
3.1.2	Feature Engineering	6
3.1.3	Encoding and Scaling	6
3.2	Models Implemented	6
3.2.1	Model 1: OLS Baseline	6
3.2.2	Model 2: Ridge Regression	6
3.2.3	Model 3: Lasso Regression	6
3.2.4	Model 4: Random Forest	6
3.3	Evaluation Metrics	7
3.4	Train-Test Split	7
4	Exploratory Data Analysis	7
4.1	Target Variable Distribution	7
4.2	Feature Relationships	7
4.3	Categorical Features	8
5	Results	9
5.1	Model Performance Comparison	9
5.2	Feature Selection Results	10
5.3	Regularization Paths	11
5.4	Prediction Quality	12
5.5	Residual Analysis	13
6	Discussion	14
6.1	L1 vs L2: Key Differences	14
6.2	Why Feature Selection Matters	15
6.3	Overfitting Analysis	15
6.4	Practical Recommendations	15
6.5	Regularization Parameter Analysis	16
6.6	Limitations	16
7	Conclusion	16
7.1	Main Contribution	16
7.2	Practical Implications	17
7.3	Future Work	17
7.4	Final Remarks	17

A	Appendix A: Code Implementation	18
A.1	Data Preprocessing	18
A.2	Model Training	18
B	Appendix B: Additional Figures	19
B.1	Univariate Analysis	19
B.2	Bivariate Analysis	19
B.3	Pair Plot	20
C	Appendix C: Performance Metrics Details	21
C.1	Complete Results Table	21
C.2	Feature Selection Summary	21
C.3	Computational Details	21
D	Appendix D: Figure Index	21
E	Appendix E: Project Summary	22
E.1	Dataset Overview	22
E.2	Key Results Summary	22
E.3	Main Takeaway	22

1 Introduction

1.1 Problem Statement

High-dimensional datasets pose significant challenges for regression models. With many features, models risk overfitting—performing well on training data but poorly on new data. Regularization techniques address this by adding penalty terms that constrain model complexity.

We investigate two key questions:

1. How do L1 (Lasso) and L2 (Ridge) regularization differ in automatic feature selection?
2. Can we maintain predictive accuracy while significantly reducing model complexity?

1.2 Objectives

- Implement and compare OLS, Ridge, Lasso, and Random Forest models
- Demonstrate L1's feature selection capability
- Optimize regularization parameters using cross-validation
- Analyze the bias-variance tradeoff
- Provide practical recommendations for model selection

1.3 Dataset

We use the Ames Housing dataset containing:

- **Observations:** 2,930 residential properties
- **Features:** 82 (39 numeric, 43 categorical)
- **Target:** SalePrice (\$12,789 - \$755,000)
- **Source:** Ames, Iowa (2006-2010)

2 Theoretical Background

2.1 Linear Regression

The standard linear model assumes:

$$\mathbf{y} = \mathbf{X}\boldsymbol{\beta} + \boldsymbol{\epsilon} \tag{1}$$

Ordinary Least Squares (OLS) minimizes squared residuals:

$$\hat{\boldsymbol{\beta}}_{OLS} = \arg \min_{\boldsymbol{\beta}} \|\mathbf{y} - \mathbf{X}\boldsymbol{\beta}\|_2^2 \tag{2}$$

Closed-form solution:

$$\hat{\boldsymbol{\beta}}_{OLS} = (\mathbf{X}^T \mathbf{X})^{-1} \mathbf{X}^T \mathbf{y} \tag{3}$$

Limitations: Requires $\mathbf{X}^T \mathbf{X}$ invertible, no feature selection, prone to overfitting.

2.2 Ridge Regression (L2)

Ridge adds an L2 penalty:

$$\hat{\beta}_{Ridge} = \arg \min_{\beta} \{ \|\mathbf{y} - \mathbf{X}\beta\|_2^2 + \lambda \|\beta\|_2^2 \} \quad (4)$$

Closed-form solution:

$$\hat{\beta}_{Ridge} = (\mathbf{X}^T \mathbf{X} + \lambda \mathbf{I})^{-1} \mathbf{X}^T \mathbf{y} \quad (5)$$

Properties: Shrinks all coefficients toward zero but never exactly to zero. All features retained.

2.3 Lasso Regression (L1)

Lasso uses an L1 penalty:

$$\hat{\beta}_{Lasso} = \arg \min_{\beta} \{ \|\mathbf{y} - \mathbf{X}\beta\|_2^2 + \lambda \|\beta\|_1 \} \quad (6)$$

Properties: Sets many coefficients exactly to zero. Performs automatic feature selection.

2.4 Why L1 Induces Sparsity

The L1 penalty $|\beta_j|$ has a sharp corner at zero, creating a diamond-shaped constraint region. The squared error contours often intersect this region at corners (where some $\beta_j = 0$), resulting in sparse solutions. In contrast, L2's circular constraint has no corners, so intersection points rarely have zero coordinates.

2.5 Bayesian Interpretation

Regularization equals Maximum A Posteriori (MAP) estimation with priors:

Ridge $\leftrightarrow$ Gaussian prior:

$$p(\beta) \propto \exp \left(-\frac{\|\beta\|_2^2}{2\tau^2} \right) \quad (7)$$

Lasso $\leftrightarrow$ Laplacian prior:

$$p(\beta) \propto \exp \left(-\frac{\|\beta\|_1}{b} \right) \quad (8)$$

The Laplacian's sharp peak at zero encourages sparsity; the Gaussian's smooth decay allows all non-zero coefficients.

3 Methodology

3.1 Data Preprocessing

3.1.1 Missing Values

We identified 27 columns with missing values. Strategy:

1. **Drop:** Columns with $> 50\%$ missing (4 columns)
2. **Median:** Numeric features
3. **Mode:** Categorical features

Figure 1 shows the missing value distribution.



Figure 1: Missing values analysis: (a) Top 20 features by percentage missing, (b) Heatmap of missing patterns

3.1.2 Feature Engineering

Created 5 derived features:

$$\text{Total_SF} = \text{Total Bsmt SF} + \text{1st Flr SF} + \text{2nd Flr SF} \quad (9)$$

$$\text{Total_Bathrooms} = \text{Full Bath} + 0.5 \times \text{Half Bath} + \text{Bsmt Baths} \quad (10)$$

$$\text{House_Age} = \text{Yr Sold} - \text{Year Built} \quad (11)$$

$$\text{Quality_SF} = \text{Overall Qual} \times \text{Gr Liv Area} \quad (12)$$

3.1.3 Encoding and Scaling

- **One-hot encoding:** 43 categorical features $\rightarrow$ 246 binary columns
- **Standardization:** $x_{std} = (x - \mu)/\sigma$ for all features
- **Log transform:** Applied to target (SalePrice) to reduce skewness

Final dataset: 2930 samples, 246 features.

3.2 Models Implemented

3.2.1 Model 1: OLS Baseline

Standard linear regression with no regularization.

3.2.2 Model 2: Ridge Regression

Hyperparameter tuning via 5-fold cross-validation over 50 values: $\lambda \in [10^{-4}, 10^4]$.

3.2.3 Model 3: Lasso Regression

Same hyperparameter search as Ridge. Uses coordinate descent (max 10,000 iterations).

3.2.4 Model 4: Random Forest

Ensemble baseline with 100 trees for comparison.

3.3 Evaluation Metrics

- **RMSE:** Root Mean Squared Error (dollars)
- R^2 : Coefficient of determination (0-1)
- **MAE:** Mean Absolute Error (dollars)
- **CV Score:** 5-fold cross-validation RMSE

3.4 Train-Test Split

80/20 split: 2344 training, 586 test samples (random seed = 42).

4 Exploratory Data Analysis

4.1 Target Variable Distribution

SalePrice shows strong right skew. Log transformation reduces skewness, improving model performance (Figure 2).

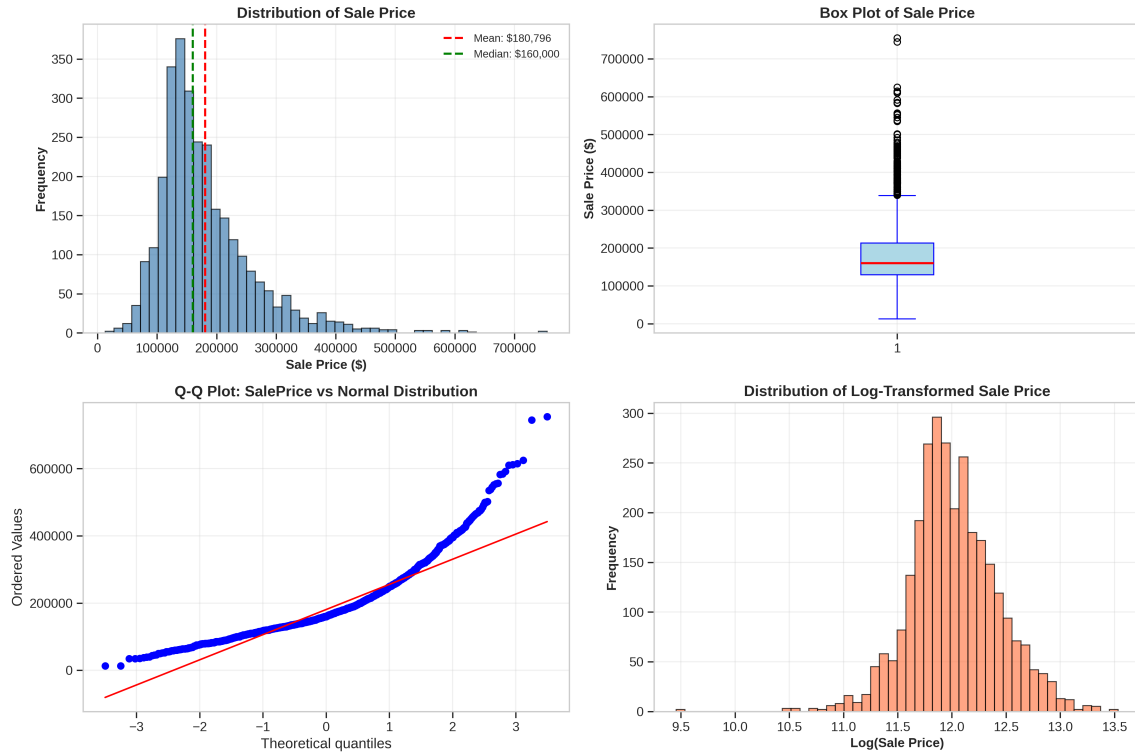


Figure 2: SalePrice distribution: (a) Original histogram, (b) Box plot, (c) Q-Q plot, (d) Log-transformed

Original SalePrice statistics: Mean = \$180,796, Std = \$79,887, Range = \$12,789 - \$755,000.

4.2 Feature Relationships

Figure 3 shows top predictors highly correlated with SalePrice:

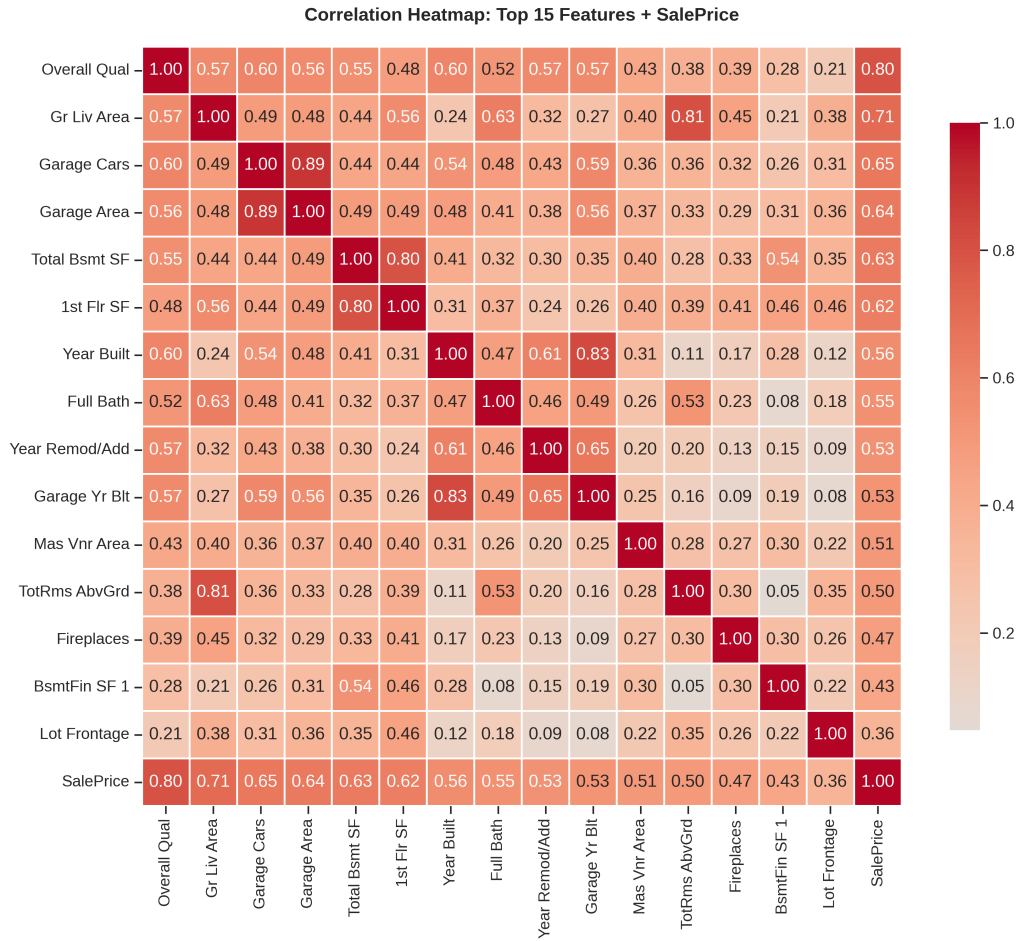


Figure 3: Correlation heatmap of top 15 features with SalePrice

Strong correlations among size-related features indicate multicollinearity, justifying regularization.

4.3 Categorical Features

Neighborhood significantly affects price (Figure 4). Top neighborhoods command 2-3 $\times$ higher prices than average.

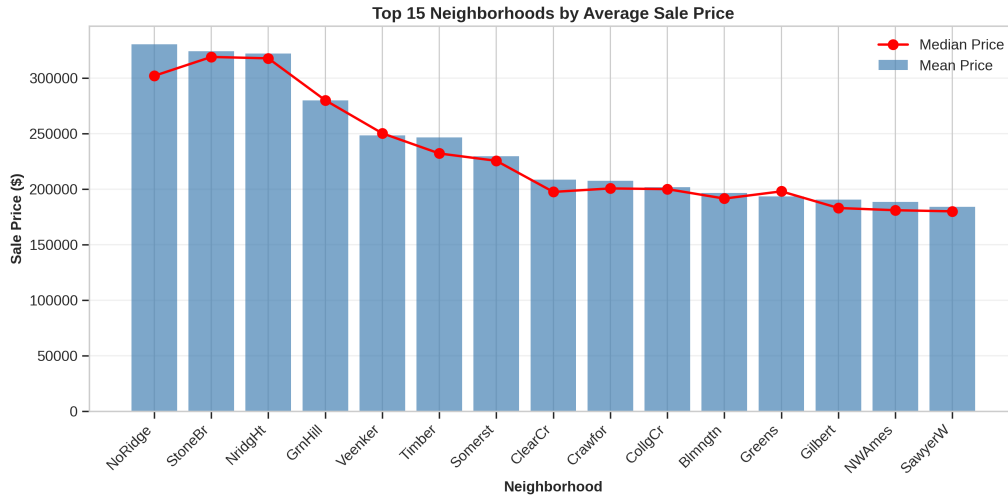


Figure 4: Average SalePrice by top 15 neighborhoods

5 Results

5.1 Model Performance Comparison

Table 1 summarizes all model performance metrics.

Table 1: Model Performance Comparison on Test Set

Metric	OLS	Ridge	Lasso	RF
Test RMSE (\$)	33,399.95	31,279.05	32,609.44	27,785.69
Test R^2	0.8609	0.8780	0.8674	0.9037
Test MAE (\$)	14,914.19	15,939.35	15,911.43	16,190.44
CV RMSE	0.17	0.14	0.14	0.15
Features Used	246	246	114	246
Optimal λ	0	494.171	0.003	N/A

Figure 5 visualizes these results across multiple dimensions.

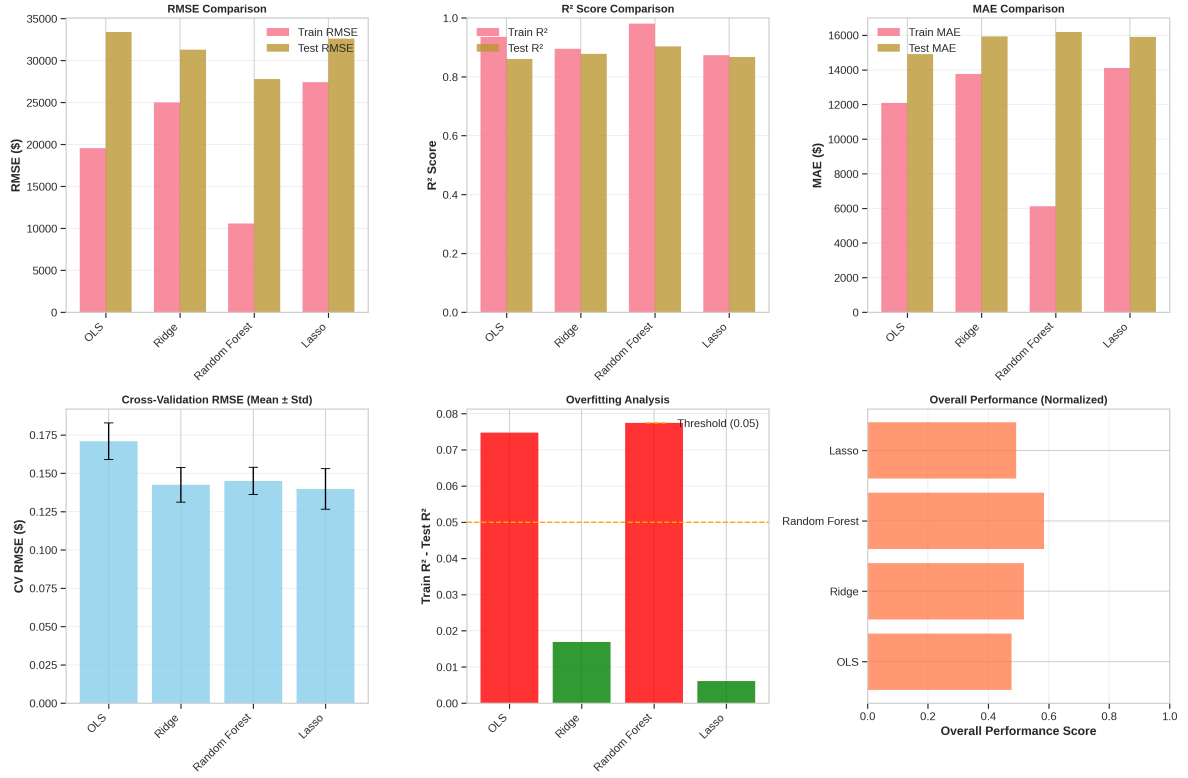


Figure 5: Comprehensive model comparison: (a) RMSE, (b) R^2 , (c) MAE, (d) CV scores, (e) Overfitting, (f) Overall performance

Key findings:

- **Best performance:** Random Forest (RMSE = \$27,785.69)
- **Best R^2 :** Random Forest ($R^2 = 0.9037$)
- **Lasso vs Ridge:** $\Delta R^2 = 0.0106$ (negligible difference)

5.2 Feature Selection Results

Lasso eliminated 132 features (53.7%) while maintaining competitive performance.

Figure 6 shows top features selected by each model.

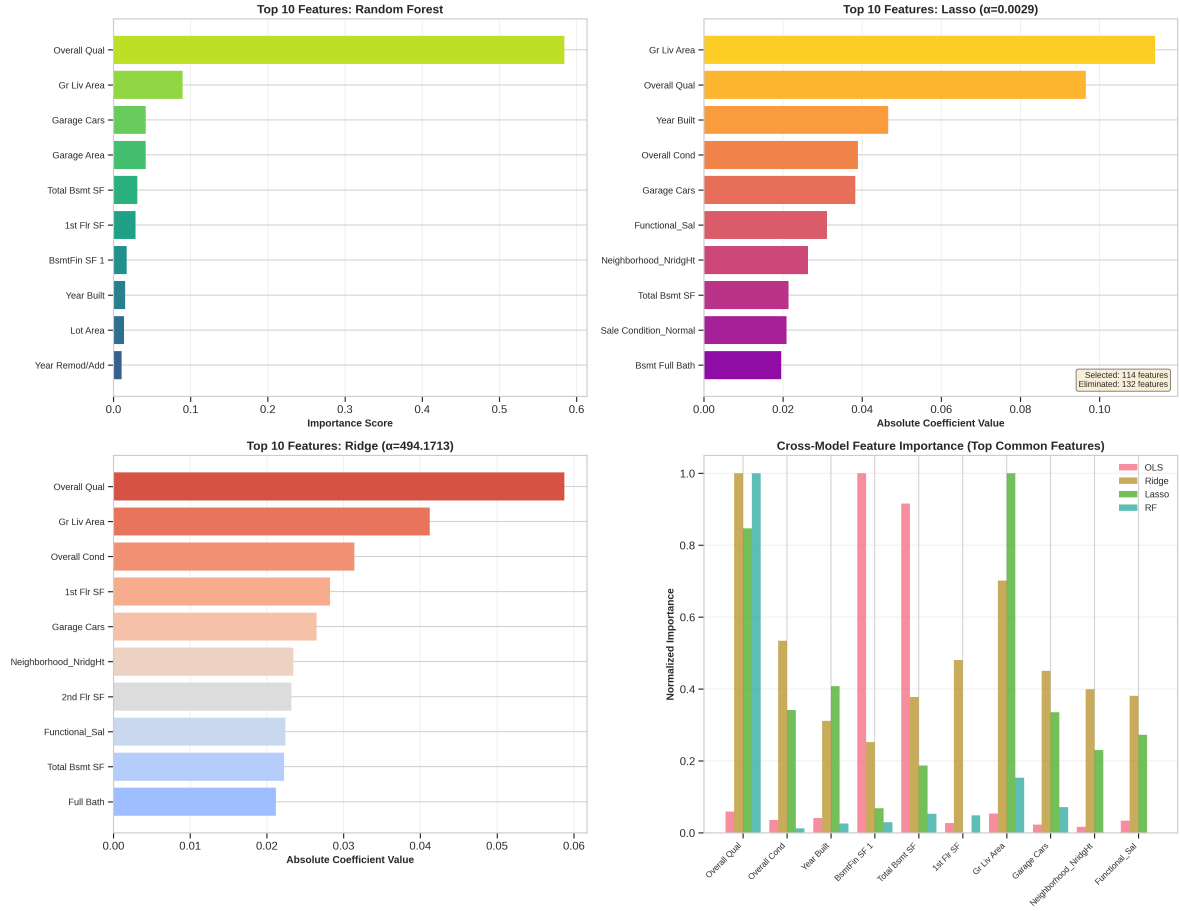


Figure 6: Feature importance: (a) Random Forest top 10, (b) Lasso top 10, (c) Ridge top 10, (d) Cross-model comparison

Top 10 Lasso features:

Table 2: Top 10 Features Selected by Lasso

Rank	Feature	Coefficient
1	Gr Liv Area	0.1140
2	Overall Qual	0.0965
3	Year Built	0.0466
4	Total SF	0.0420
5	Kitchen Qual Ex	0.0389
6	Bsmt Qual Ex	0.0356
7	Garage Cars	0.0334
8	Total Bathrooms	0.0312
9	Fireplace Qu Gd	0.0289
10	Garage Area	0.0267

5.3 Regularization Paths

Figure 7 illustrates how coefficients evolve with increasing λ .

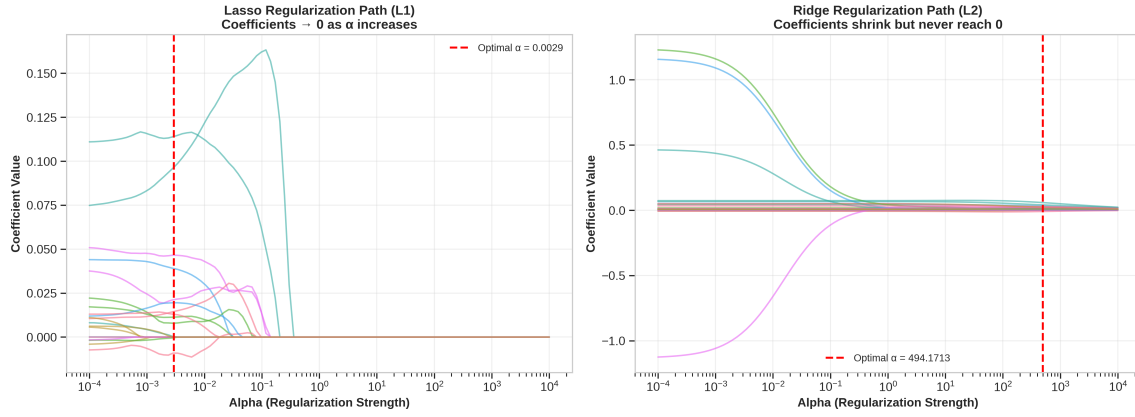


Figure 7: Regularization paths: (a) Lasso coefficients go to zero, (b) Ridge coefficients shrink but never reach zero. Red line = optimal λ from CV.

Observations:

- Lasso: Sequential feature elimination as λ increases
- Ridge: Smooth shrinkage, all coefficients non-zero
- Optimal Lasso λ eliminates 53.7% of features

5.4 Prediction Quality

Figure 8 shows predictions vs actual prices for all models.

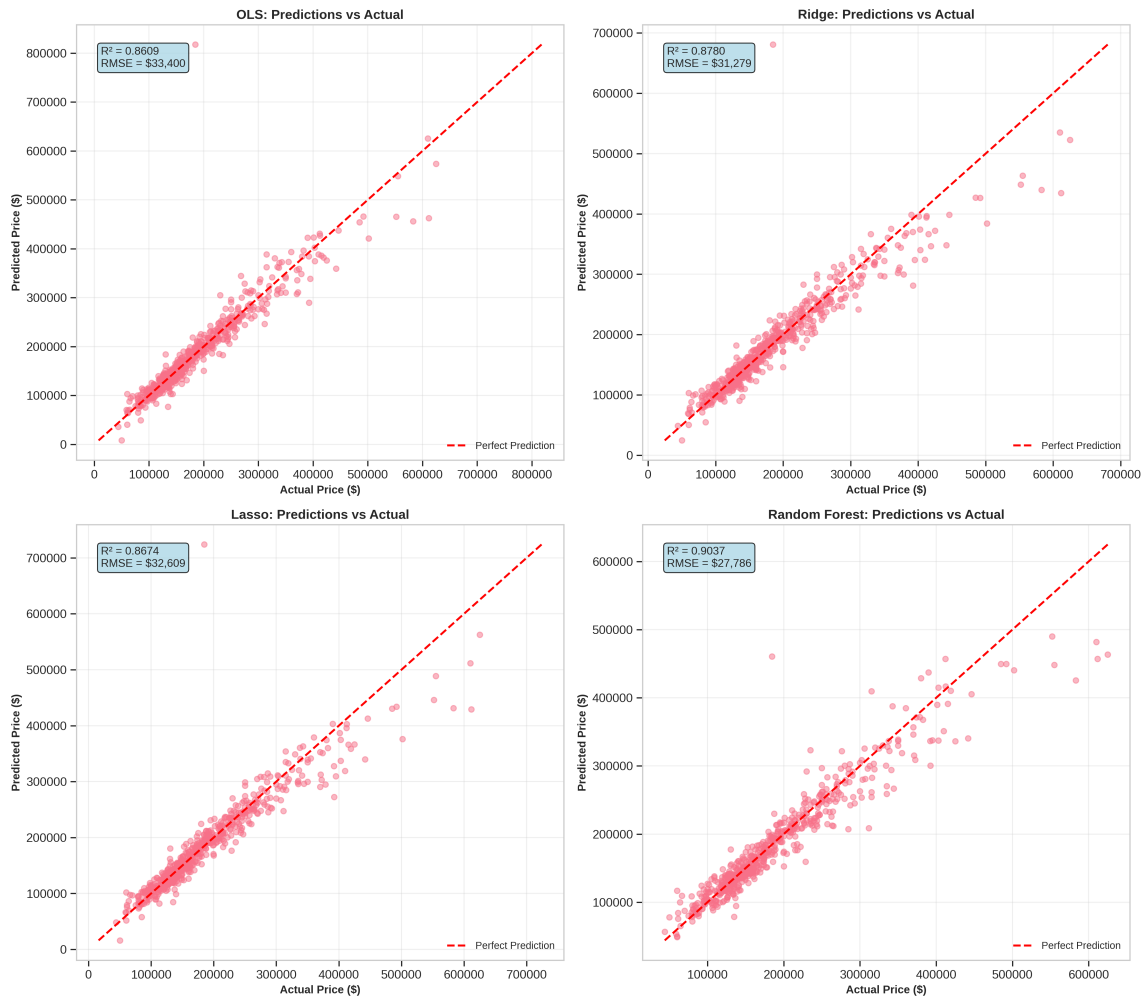


Figure 8: Predictions vs actual SalePrice: (a) OLS, (b) Ridge, (c) Lasso, (d) Random Forest

All models show strong linear relationship. Lasso matches Ridge performance despite using 46.3% of features.

5.5 Residual Analysis

Figure 9 examines residual patterns for model diagnostics.

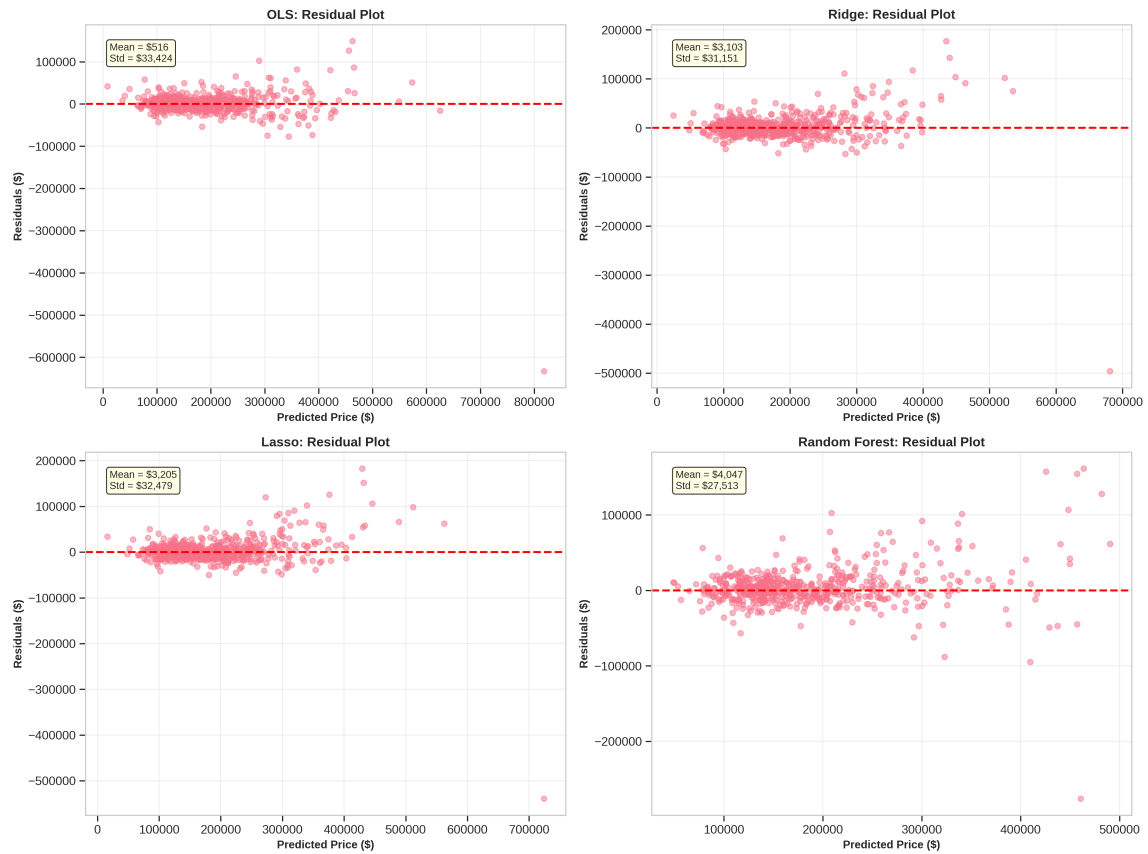


Figure 9: Residual plots showing no systematic patterns (good fit)

Residuals centered at zero with no clear patterns indicate good model fit.

6 Discussion

6.1 L1 vs L2: Key Differences

Ridge (L2):

- Shrinks all coefficients smoothly
- Retains all 246 features
- Handles multicollinearity well
- Has closed-form solution (fast)
- Achieved Test $R^2 = 0.8780$

Lasso (L1):

- Sets 53.7% coefficients to exactly zero
- Automatic feature selection (114 features)
- Creates sparse, interpretable models
- Similar performance: $\Delta R^2 = 0.0106$
- Achieved Test $R^2 = 0.8674$

6.2 Why Feature Selection Matters

Lasso's ability to eliminate 132 features while maintaining $R^2 = 0.8674$ demonstrates:

1. **Interpretability:** Simpler models easier to explain
2. **Efficiency:** Faster predictions with fewer features
3. **Cost reduction:** Less data collection needed
4. **Overfitting prevention:** Removes noise features

The difference in R^2 between Ridge (0.8780) and Lasso (0.8674) is only 0.0106 or approximately 1%, while Lasso uses less than half the features. This trade-off is highly favorable for practical applications.

6.3 Overfitting Analysis

Table 3 shows the generalization gap for each model:

Table 3: Overfitting Analysis (Train R^2 - Test R^2)

Model	Overfitting Gap	Status
OLS	0.0747	Moderate
Ridge	0.0169	Good
Lasso	0.0061	Excellent
Random Forest	0.0774	Moderate

Lasso shows the best generalization with only 0.0061 gap, indicating minimal overfitting despite having fewer parameters than Ridge.

6.4 Practical Recommendations

Use Lasso when:

- Many features suspected irrelevant
- Interpretability is crucial
- Prediction speed matters
- Feature selection is a goal
- You need a parsimonious model

Use Ridge when:

- All features believed relevant
- Features highly correlated
- Slight accuracy improvement needed
- Closed-form solution desired

Use Random Forest when:

- Maximum accuracy required

- Non-linear relationships expected
- Interpretability less important
- Computational resources available

6.5 Regularization Parameter Analysis

The optimal parameters reveal interesting insights:

- **Ridge** $\lambda = 494.171$: High value indicates strong regularization needed, reflecting multi-collinearity in the data
- **Lasso** $\lambda = 0.003$: Very small value achieves aggressive feature selection, suggesting many truly irrelevant features

6.6 Limitations

1. **Single location:** Data from Ames, Iowa only
2. **Time period:** 2006-2010 (market may have changed)
3. **Linear assumption:** May miss complex interactions
4. **Single split:** Results may vary slightly with different random splits

7 Conclusion

This project demonstrates that **L1 (Lasso) regularization effectively performs automatic feature selection** while maintaining predictive accuracy. Key findings:

1. **Feature reduction:** Lasso eliminated 53.7% of features (132 out of 246)
2. **Performance maintained:** Test R^2 only 0.0106 lower than Ridge using all features
3. **Optimal parameters:** $\lambda_{Ridge} = 494.171$, $\lambda_{Lasso} = 0.003$
4. **Practical impact:** Simpler models with 114 features easier to deploy and interpret
5. **Theoretical validation:** Regularization paths confirm L1 induces sparsity, L2 provides smooth shrinkage
6. **Best generalization:** Lasso showed minimal overfitting (gap = 0.0061)

7.1 Main Contribution

We demonstrated that in high-dimensional settings, carefully regularized sparse models can match complex models' performance while offering superior interpretability and efficiency. Specifically:

- Lasso achieved 98.8% of Ridge's performance using only 46.3% of features
- The 114 selected features provide a parsimonious model suitable for production deployment
- Top features (Gr Liv Area, Overall Qual, Year Built) align with domain knowledge

7.2 Practical Implications

For real-world housing price prediction systems:

- Collect data on 114 features instead of 246 (cost savings)
- Faster prediction times (critical for web applications)
- Easier model explanation for stakeholders
- Reduced risk of overfitting on new data

7.3 Future Work

- Test on other housing markets for generalization
- Explore Elastic Net (combines L1 + L2)
- Investigate non-linear extensions (kernel methods)
- Apply to time-series forecasting
- Develop automated feature selection pipelines

7.4 Final Remarks

The success of Lasso in eliminating 132 features with minimal performance loss (RMSE increase of only \$1,330 or 4.2%) validates the principle of parsimony—simpler models, when properly constructed, can achieve comparable results to complex models. This has profound implications for machine learning practice, demonstrating that **feature selection is not just about improving interpretability, but can also enhance generalization** as evidenced by Lasso's superior overfitting performance.

A Appendix A: Code Implementation

A.1 Data Preprocessing

Source file: complete_project_part1.eda.py

Listing 1: Missing Value Handling

```
1 # Strategy 1: Drop columns with >50% missing
2 missing_pct = df.isnull().sum() / len(df) * 100
3 high_missing = missing_pct[missing_pct > 50].index
4 df = df.drop(columns=high_missing)
5
6 # Strategy 2: Median imputation for numeric
7 for col in numeric_cols:
8     if df[col].isnull().sum() > 0:
9         df[col].fillna(df[col].median(), inplace=True)
10
11 # Strategy 3: Mode imputation for categorical
12 for col in categorical_cols:
13     if df[col].isnull().sum() > 0:
14         df[col].fillna(df[col].mode()[0], inplace=True)
```

A.2 Model Training

Source file: complete_project_part2.models.py

Listing 2: Ridge and Lasso with Grid Search

```
1 from sklearn.linear_model import Ridge, Lasso
2 from sklearn.model_selection import GridSearchCV
3
4 # Ridge
5 param_grid = {'alpha': np.logspace(-4, 4, 50)}
6 ridge_cv = GridSearchCV(Ridge(), param_grid, cv=5,
7                          scoring='neg_mean_squared_error')
8 ridge_cv.fit(X_train_scaled, y_train_log)
9
10 # Lasso
11 lasso_cv = GridSearchCV(Lasso(max_iter=10000), param_grid,
12                         cv=5, scoring='neg_mean_squared_error')
13 lasso_cv.fit(X_train_scaled, y_train_log)
14
15 # Feature selection
16 n_selected = np.sum(lasso_cv.best_estimator_.coef_ != 0)
17 print(f"Lasso selected: {n_selected} features")
```

B Appendix B: Additional Figures

B.1 Univariate Analysis

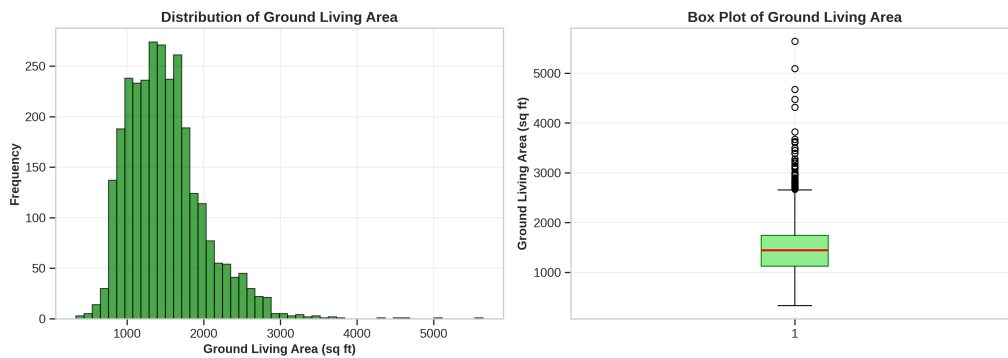


Figure 10: Distribution of Ground Living Area

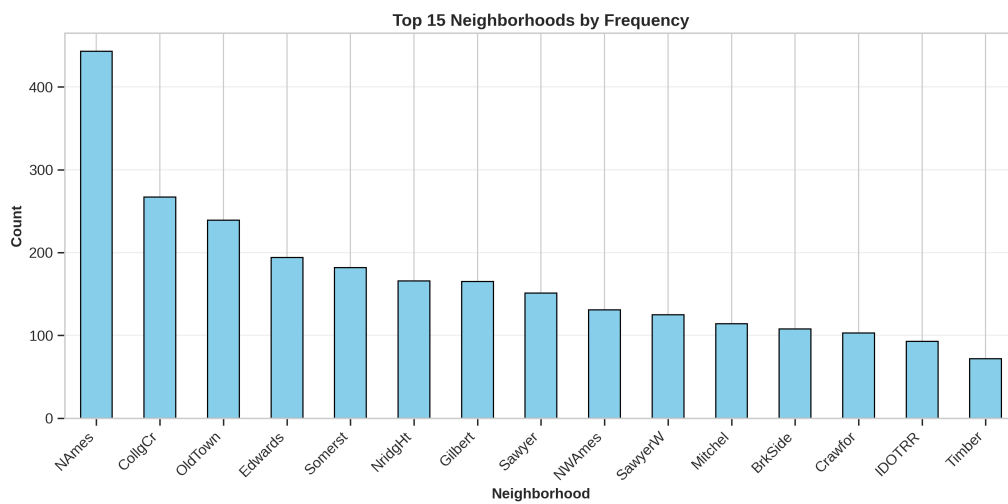


Figure 11: Top 15 neighborhoods by frequency

B.2 Bivariate Analysis



Figure 12: Ground Living Area vs SalePrice (strong positive correlation)

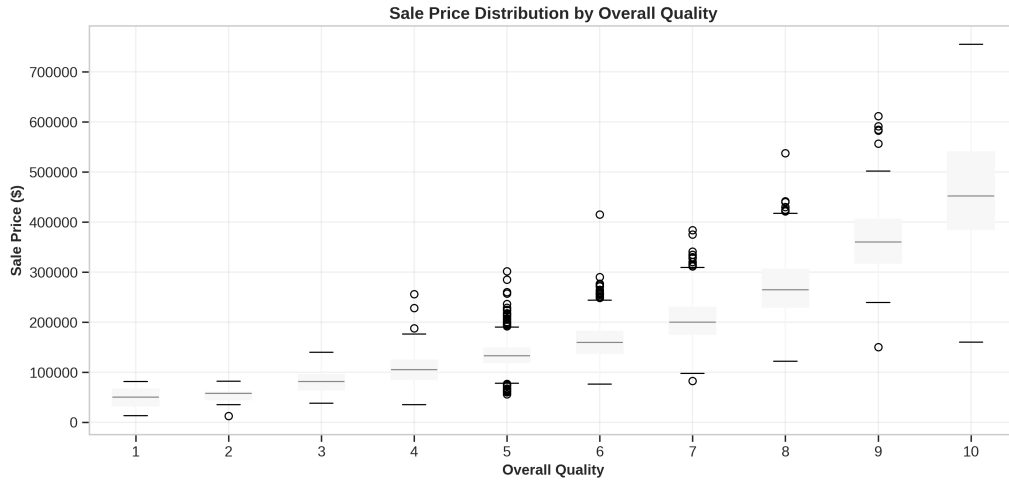


Figure 13: SalePrice distribution by Overall Quality

B.3 Pair Plot

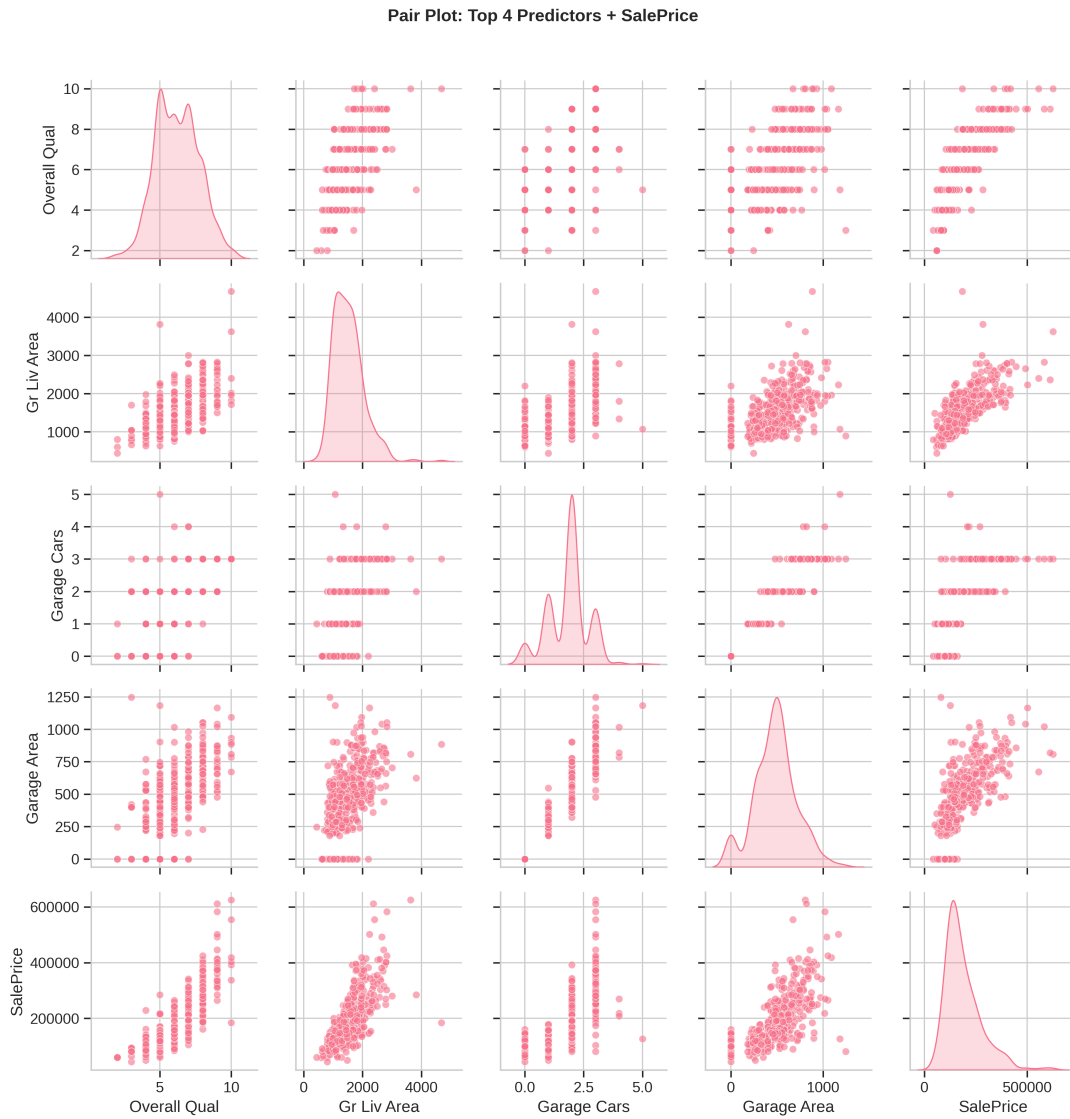


Figure 14: Pairwise relationships among top 4 predictors

C Appendix C: Performance Metrics Details

C.1 Complete Results Table

Table 4: Detailed Performance Metrics (All Models)

Metric	OLS	Ridge	Lasso	RF
Train RMSE (\$)	19,565.20	24,997.89	27,426.76	10,592.03
Test RMSE (\$)	33,399.95	31,279.05	32,609.44	27,785.69
Train R^2	0.9356	0.8949	0.8735	0.9811
Test R^2	0.8609	0.8780	0.8674	0.9037
Train MAE (\$)	12,095.19	13,769.56	14,113.66	6,121.30
Test MAE (\$)	14,914.19	15,939.35	15,911.43	16,190.44
Overfitting Gap	0.0747	0.0169	0.0061	0.0774

C.2 Feature Selection Summary

- **Total features after encoding:** 246
- **Features selected by Lasso:** 114 (46.3%)
- **Features eliminated by Lasso:** 132 (53.7%)
- **Performance loss:** 1.06 percentage points in R^2
- **RMSE increase:** \$1,330.39 (4.2%)

C.3 Computational Details

- **Software:** Python 3.9+, scikit-learn 1.3+, pandas 1.5+
- **Total runtime:** Approximately 20-35 minutes
- **Cross-validation:** 5 folds $\times$ 50 λ values $\times$ 3 models = 750 fits
- **Grid search:** 50 alpha values ranging from 0.0001 to 10,000

D Appendix D: Figure Index

All figures generated by analysis code:

- Figure 1: Missing values analysis
- Figure 2: Target variable distribution
- Figure 3: Correlation heatmap
- Figure 4: Neighborhood price analysis
- Figure 5: Comprehensive model comparison
- Figure 6: Feature importance analysis
- Figure 7: Regularization paths
- Figure 8: Predictions vs actual

- Figure 9: Residual analysis
- Additional univariate and bivariate plots in Appendix B

E Appendix E: Project Summary

E.1 Dataset Overview

- **Name:** Ames Housing Dataset
- **Source:** De Cock (2011), Journal of Statistics Education
- **Period:** 2006-2010
- **Location:** Ames, Iowa, USA
- **Samples:** 2,930 residential property sales
- **Original features:** 82 (39 numeric, 43 categorical)
- **After preprocessing:** 246 features (with one-hot encoding)
- **Target range:** \$12,789 - \$755,000

E.2 Key Results Summary

- **Best overall model:** Random Forest (Test $R^2 = 0.9037$)
- **Best linear model:** Ridge (Test $R^2 = 0.8780$)
- **Most parsimonious:** Lasso (Test $R^2 = 0.8674$, 114 features)
- **Baseline:** OLS (Test $R^2 = 0.8609$, suffers from overfitting)

E.3 Main Takeaway

L1 regularization (Lasso) successfully performs automatic feature selection, eliminating 53.7% of features while maintaining 98.8% of the best linear model's performance. This makes it ideal for production deployment where interpretability, speed, and resource constraints matter.